

Mix-supervised Learning: A Self-Learning Strategy for Facilitating Data Utilization and Robust Representation

Qinglei Li, Nanyi Wang, Qi Wang*, Member, Chenyang Wu, Jiebo Luo *IEEE Fellow*

Abstract—Deep learning has significantly advanced image classification, but it traditionally relies on labeled data, limiting the model’s ability to learn from “unseen” or unlabeled samples. To address this, we propose the Mix-supervised Learning paradigm, inspired by human learning, which allows models to acquire new knowledge from unlabeled data. In Mix-supervised Learning, the model can not only learn from labeled data but also improve its performance by leveraging out-of-training samples, similar to how humans adapt and self-improve, thus facilitating data utilization. To implement Mix-supervised learning, we propose a training phase called self-learning. Our self-learning integrates several core components: 1) a Class-Balanced Cross-Entropy Loss that handles class imbalance in pseudo-label generation, 2) a Model Rewind mechanism that maintains training stability by reverting detrimental parameter updates through historical snapshot comparison, and 3) pseudo-prototype-based feature regularization (ensuring semantic consistency at the feature level) and adaptive feature calibration (refining pseudo-labels via prototype distribution alignment). Extensive experiments on benchmark datasets show that our approach outperforms traditional methods in terms of accuracy and generalization, by making better use of all available data and enhancing the model’s robustness when processing unseen data.

I. INTRODUCTION

The development of deep learning has led to substantial advancements in various fields, particularly in computer vision. Image classification, being a fundamental task in computer vision [48], is crucial for a variety of downstream tasks, such as object detection and image segmentation [18], [46], [50]. For CNNs in image classification, the process typically begins with the extraction of features from the samples using convolution kernels or custom-designed feature extractors [47]. Then, the model analyzes the features and outputs the confidence scores for each class via a linear layer. A sample is classified as belonging to the class with the highest confidence score [22], [41]. To train the model, the correct class is predicted, and the confidence score for the correct class will equal near 100%. The process is crucial in ensuring the effectiveness and accuracy of the model, as it helps the model learn sufficient knowledge and gradually become closer to reality with each training session.

In particular, for the current image classification methods, a training set is adopted to guide the training of the model

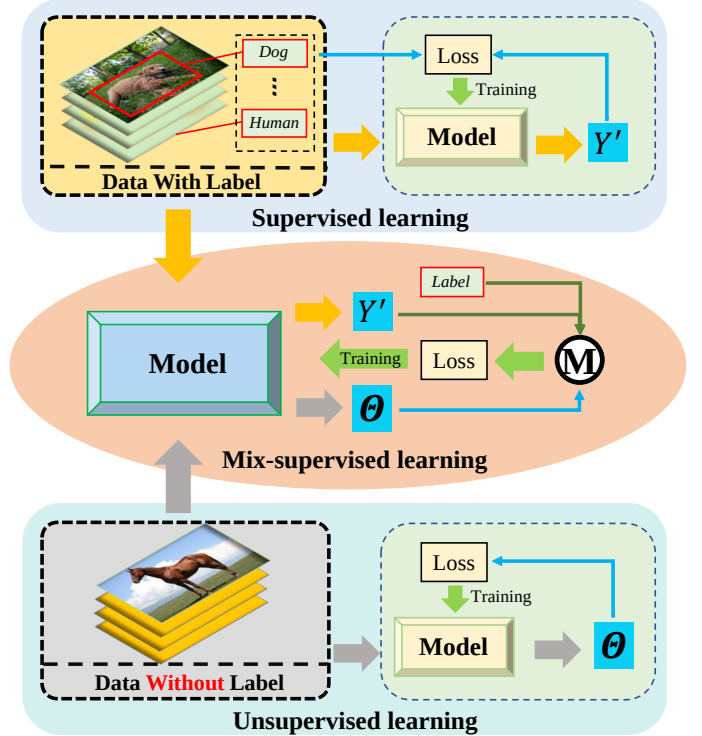


Fig. 1. The paradigm of mixed supervised learning: a combination of supervised and unsupervised learning, belonging to the category of semi-supervised learning. The learning method is analogous to how humans simultaneously learn from both “known” data and “unknown” data, and is found to be effective.

through images and labels [16]. During the training process, the validation set is used to evaluate the model’s performance and convergence after each training epoch. After training, the model’s training mode is switched off, and certain layers such as normalization and dropout layers are disabled [1], [28]. A test set is used to evaluate the model’s generalization ability and performance. Unlike the training set, the validation and test sets do not provide labels to the model during training, which means that the model’s training solely depends on the labeled data in the training set. Overall, the model’s perception of the unknown samples from the validation or test set is not considered.

Current learning paradigms often rigidly separate supervised and unsupervised learning, limiting both data utilization and model adaptability to real-world scenarios with emerging

Q. Li, N. Wang, Q. Wang and C. Wu are with the State Key Laboratory of Public Big Data, College of Computer Science and Technology, Guizhou University, Guiyang 550025, China.

J. Luo is with the Department of Computer Science, University of Rochester, Rochester, NY 14627, USA.

Our code is available at: <https://github.com/Sheerange/Self-Learning>.

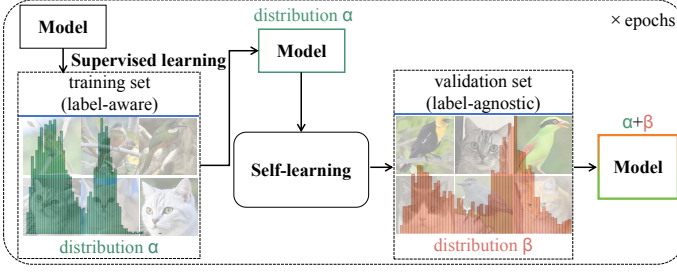


Fig. 2. The core of Mix-supervised Learning. In each training epoch, the model learns the data patterns of distribution α from the training set in a supervised manner. The mix-supervised learning further incorporates a process of perceiving new data to acquire additional knowledge. This process employs self-learning to extract the data patterns of another distribution β from the label-agnostic validation set. By integrating both distributions, the mix-supervised approach broadens the model’s learned data pattern, thereby enhancing its generalization capability.

unseen data and distribution shifts. Inspired by the fluid integration of instruction and exploration in human learning, we propose Mix-Supervised Learning - a framework that strategically combines supervised learning on labeled data with unsupervised learning on unlabeled data (Fig. 1). Semi-supervised learning [34] aims to augment limited labeled data with in-distribution unlabeled training samples to reduce annotation costs, whereas Mix-Supervised Learning redefines the validation set as an out-of-distribution adaptation resource, thereby maximizing the utility of all available data. The former addresses label scarcity within fixed distributions, while the latter tackles real-world deployment challenges where models must continuously adapt to shifting data environments. The mix-supervised learning mirrors how humans augment formal instruction with empirical experience [21], but is implemented through a label-agnostic learning mechanism that systematically exploits validation-set patterns without label dependency.

To gain a better understanding of new information, humans can reap cognitive benefits through predicting and evaluating confidence levels. When encountering new information, humans first generate an initial prediction with low confidence. Then, through theoretical derivation and analysis, their confidence level may increase or decrease, eventually reaching 0% or 100%. Regardless of the confidence level trend, humans can always gain new insights and self-improvement. Similarly, in model training, reinforcement learning, proposed by Mnih et al. [20], is also modeled after human learning, where the model learns through feedback from the environment [16], [35]. However, the method is limited to tasks with defined rules and is not effective for tasks with uncertain rules, such as image classification. To overcome the limitation, Curtis et al. [21] proposed confidence learning to improve the model’s learning ability, but this method does not improve the model’s original judgment. For self-supervised learning, normal methods train the model with samples selected based on confidence thresholds but do not allow the model to escape from fixed and reliable samples, leading to noisy data and negative training results. To bridge this gap while preserving the human-like learning benefits of confidence-based adaptation, we propose a self-learning training phase that implements mix-supervised

learning. The self-learning training step enables the model to acquire knowledge from inference as well, expanding the knowledge scope of the model, as shown in Fig. 2.

In this study, we develop an integrated suite of methodologies under the self-learning strategy. First, we introduce the definition of mix-supervised learning, illustrate the meaning of mix-supervised learning and explain why mix-supervised learning is important. Then, Unlike traditional semi-supervised learning methods where standard cross-entropy loss applies, the mix-supervised training mechanism requires specialized adaptation, prompting our proposed Class Balanced Cross Entropy Loss. Additionally, we introduce a method called Model Rewind, which reverts the model parameters to their state before the erroneous update, in order to correct the model’s convergence direction. Finally, we formalize the pseudo-prototype concept as feature anchors, proposing: (1) a pseudo-prototype-based feature regularization method that preserves semantic consistency at the feature level, and (2) a pseudo-prototype-guided feature calibration approach to enhance the quality of pseudo-labels. The evaluation is conducted on various benchmark datasets, including CIFAR-100, tiny-ImageNet. The suggested self-learning outperforms the existing normal learning method in terms of accuracy and generalization ability. These results confirm that the method we propose offers a more reasonable approach to the learning paradigm of model training, making full use of all available data during the training process, similar to human learning.

Overall, the main contributions of this work are as follows:

- We propose mix-supervised learning, a paradigm that implements self-learning by strategically leveraging limited out-of-distribution validation data through label-agnostic tuning, thereby enabling autonomous knowledge refinement for unseen data scenarios.
- We design a CB-CE Loss to address the issue of imbalanced pseudo-label distribution in the self-learning process, and introduce Model Rewind to guide the model’s learning towards the correct direction.
- We introduce pseudo-prototype-based feature regularization, which preserves semantic consistency during feature transformation, and pseudo-prototype-guided feature calibration to improve pseudo-label reliability.

II. RELATED WORK

For our study, we draw inspiration from the human learning method and the studies in various fields, including supervised learning, unsupervised learning, and semi-supervised learning. Hence, we briefly mention these three topics.

A. Supervised Learning

Supervised learning is a type of machine learning strategy that relies on labeled data to train a model that can accurately predict the output of new, unlabeled data. In supervised learning, the input and output data are predefined, and the goal is to find a relationship or pattern between the input and output data. The process starts with training with known samples to obtain an optimal model [38]; then, the model is applied to new data to produce the output. The primary emphasis of supervised

learning is the development of classifiers and the establishment of corresponding weightings [36]. It is commonly used for classification but has poorer interpretability than unsupervised learning. Examples of classical supervised learning algorithms include linear regression, neural networks, decision tree, support vector machine (SVM), k-nearest neighbor (KNN), and naive Bayes [19], [23], [31]. This paper proposes mixed supervised learning, with supervised learning being a crucial component, and a model optimization algorithm based on supervised learning that can incorporate the differences in knowledge between the labeled and unlabeled samples.

B. Unsupervised Learning

Unsupervised learning algorithms derive conclusions from unlabeled training data, a type of machine learning task. These methods are used to uncover the intrinsic structure of data based on the relationships between variables [15], [24], [51]. Unlike supervised learning, unsupervised learning algorithms do not have access to pre-labeled output data, and the goal is to identify patterns or groupings in the input data without prior knowledge of the output variable [3], [45]. Unsupervised learning algorithms use statistical methods to uncover the intrinsic structure of the data based on the relationships between variables. The focus of unsupervised learning is clustering and dimensionality reduction, mainly to estimate density by identifying statistical values that describe the data [52]. The common unsupervised learning algorithms include principal component analysis (PCA), autoencoder algorithms (AE), generative adversarial network (GAN), self-organizing map (SOM), and singular value decomposition (SVD) [4], [11], [13], [26], [33]. In this study, mix-supervised learning is performed using both labeled and unlabeled samples to optimize the training of the model, with unsupervised learning playing an important role in the mixed supervision process, which can help models form new knowledge from unknown samples.

C. Semi-supervised Learning

Semi-supervised learning utilizes a small amount of labeled data together with a large amount of unlabeled data to improve learning accuracy, where the unlabeled data helps capture the underlying data structure [42]. In a typical machine learning problem, there is a dataset containing both input data and their corresponding labeled output data [5], [54]. However, labeling data can be time-consuming, expensive, and sometimes even impossible. In semi-supervised learning, we utilize both labeled and unlabeled data to build models that can improve accuracy, reduce overfitting, and make better predictions [14], [29]. In this method, a portion of the training data is labeled, while the rest is unlabeled [7]. The amount of unlabeled data is typically larger than that of labeled data [6]. The learning machine uses the knowledge gained from the labeled data as a basis and gradually refines it with the distribution of the unlabeled data, ultimately determining the class of the test samples [53]. The method can produce acceptable classification results even with a small amount of labeled data, as the distribution

of the data is not completely random [2], [12]. Some classical semi-supervised learning algorithms include expectation maximization (EM) and the low-density separation algorithm [32], [49]. Semi-supervised learning relies on abundant in-distribution unlabeled training data, whereas mix-supervised Learning intentionally employs limited out-of-distribution validation data through label-agnostic self-learning, explicitly targeting distribution shift robustness.

III. METHODOLOGY

A. Mix-supervised Learning

1) *Definition of Mix-supervised Learning*: Mix-supervised learning refers to a learning method that combines both supervised and semi-supervised learning to improve the performance of the model. The concept of mix-supervised learning is inspired by human learning, where people learn from both structured information, such as textbooks, and through personal experience and self-discovery. Specifically, the model can effectively utilize evaluation data so that the evaluation data is no longer solely used for assessing model performance, while ensuring that the model does not "peek" at the evaluation data. The model can "perceive" the content of the evaluation data without seeing their labels.

2) *Why Mix-supervised-Learning is Important*: Our experimental results show that mix-supervised learning is able to work on image classification across a variety of model architectures and multiple datasets. This proposes a feasible "optimization" paradigm for the conventional learning approach of training on the training set and validating on the validation set. In the past, validation data is solely used for evaluation purposes, overlooking its inherent learnable value as data. If there is an appropriate method that allows the model to train on validation data without "peeking" at it, this would not only preserve the evaluation role of the validation data but also enable the model to learn additional feature patterns.

3) *Self-Learning*: The mix-supervised learning proposed in this paper represents a novel training paradigm. In this work, we further introduce an additional training step aligned with the concept of mix-supervised learning: self-learning. Specifically, building upon the traditional training process, we incorporate an additional step of self-learning. This process is conducted on the validation dataset, where the model attempts to acquire knowledge from unseen data while the labels remain inaccessible.

However, there is a risk that the model may assimilate incorrect or misleading information. To address this, we leverage the inherent verifiability of the validation data and introduce a rewind mechanism, which aims to steer the learning process away from such detrimental conditions. Consequently, from an expected value perspective, self-learning proves to be an effective strategy, capable of enhancing the model's performance, particularly in terms of its generalization capabilities.

B. Training with Pseudo-Labels

1) *Class Balanced Cross Entropy Loss*: Cross-entropy loss is a widely used loss function, particularly for classification

tasks. It measures the difference between the predicted probability distribution and the actual distribution. For multi-class classification, the cross-entropy loss is defined as follows:

$$L_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(p_{i,c}), \quad (1)$$

where N is the number of samples, C is the number of classes, $y_{i,c}$ is a binary indicator if class label c is the correct classification for sample i , and $p_{i,c}$ is the predicted probability that sample i belongs to class c .

In semi-supervised learning, the cross-entropy loss function is also widely used. However, the standard cross-entropy loss function is not suitable for training a model on unlabeled validation data. This is because the size of the unlabeled validation data varies, and there might be significant class imbalance. In the early stages of training, the model's recognition ability is still immature, resulting in a very small number of pseudo-labeled samples. The standard cross-entropy loss balances based on batch size, meaning that even a small number of pseudo-labeled samples can contribute a loss value of the same magnitude as that from a large number of normal samples. This causes the model to over-optimize on this small set of pseudo-labeled samples, which might include incorrect labels. Additionally, the initial pseudo-labels assigned by the model can bias it towards those categories, leading to more pseudo-labels being assigned to these categories subsequently. This reduces the likelihood of other categories receiving pseudo-labels, exacerbating class imbalance.

To address the issues of excessively large loss values and class bias, we remove batch balancing from the original cross-entropy loss and balance the loss magnitude across classes. We call it Class Balanced Cross Entropy Loss, which is formulated as follows:

$$L_{CB-CE} = -\frac{1}{C} \sum_{c=1}^C \frac{1}{N_c} \sum_{i=1}^{N_c} y_{i,c} \log(p_{i,c}), \quad (2)$$

where N_c is the number of samples with label c . For training on the labeled dataset, we still use the standard cross-entropy loss. We only apply the CB-CE Loss for self-learning on the unlabeled validation data. The CB-CE Loss effectively mitigates the challenges associated with class imbalance in pseudo-labeling and alleviates the limitation of insufficient pseudo-labeled samples.

2) *Model Rewind*: The performance of standard semi-supervised learning frameworks is inherently constrained by the difficulty in quantitatively assessing pseudo-label quality - a factor that critically determines model optimization effectiveness. In contrast to conventional approaches, our method performs immediate performance validation on the complete validation set following each semi-supervised training iteration on unlabeled validation data. By comparing the performance before and after updating with the current batch of pseudo-labeled samples, we can roughly estimate the quality of the pseudo-labels assigned in the current round, as shown in Fig.3. Furthermore, if we determine that the quality of the current pseudo-labels is low, we will rewind the model to its state

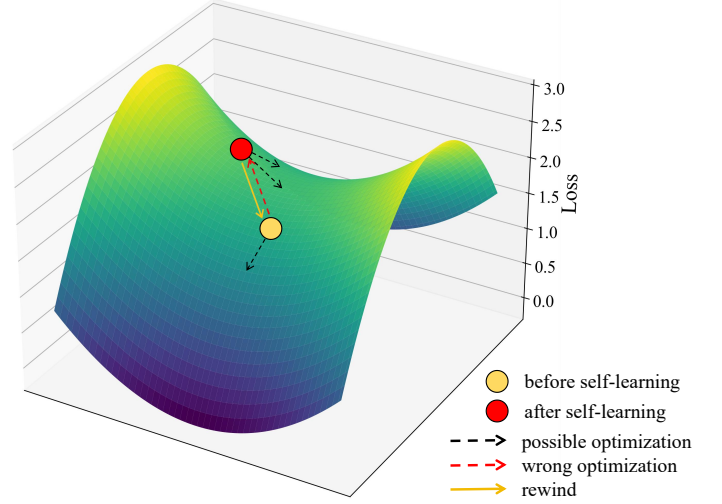


Fig. 3. The diagram illustrates a failed self-learning attempt. Initially, the model resides at the yellow circle position. A self-learning step with excessive incorrect pseudo-labels pushes the model to a higher point on the loss curve. At this point, the model's next update could follow multiple suboptimal directions, potentially trapping it in a worse local optimum. The rewind mechanism mitigates this risk by rolling back the model parameters upon detecting performance degradation after self-learning.

before the current batch of pseudo-labeled samples was used for updates:

$$acc(M_{after}) - acc(M_{before}) > \tau, \quad (3)$$

where M_{after} denotes the model after the parameter update, M_{before} represents the model prior to the update, $acc(\cdot)$ is an accuracy metric function, and τ is a predefined threshold.

In addition to reverting the model's parameters to a previous state, the optimizer's state should also be reverted to eliminate momentum and other accumulated information.

C. Pseudo-Prototype and Consistency Regularization

Learning with pseudo-labeled samples is intuitive and straightforward, but the correctness of pseudo-labels cannot be guaranteed. In contrast, consistency regularization methods train models without assigning pseudo-labels by requiring consistent outputs for semantically preserved transformations of the same input. Traditional approaches typically achieve such transformations through image augmentations (e.g., flipping, cropping, color shifting), but feature-level consistency regularization has been rarely explored—designing semantically preserved feature transformations is highly challenging due to the high-dimensional and abstract nature of feature space. However, in self-learning scenarios where unlabeled data is relatively scarce, the feature patterns learned from labeled data exhibit good discriminability and reliability. Thus, we can leverage class prototypes learned from labeled data as centroids, treating the interaction process between features and their corresponding class centroids as semantically preserved feature transformations.

1) *Pseudo-Prototypes*: Some related works take additional steps to extract prototypes from the dataset, including mean vectors, clustering algorithms, etc., which introduce extra time

costs. In our work, we do not design any additional steps to extract prototypes from the dataset. Instead, we use a "natural prototype" that already has class representation capabilities on the dataset, which is the weight of the linear classifier, referring to this as the pseudo-prototype.

In image classification tasks, the final layer of the model is almost always a linear layer, which maps the feature vector to the classes in order to compute the scores for each class. Let $W = [w_1, w_2, \dots, w_C] \in R^{C \times d}$ represents the weight matrix inside the linear classifier, where C and d indicate the number of classes and dimension of feature vector, respectively. In addition, the feature vector of the input image x after processing by the network's backbone is denoted as $f(x)$. Then the *Score* of input x corresponding to each class c are calculated as:

$$S_c = w_c \cdot f(x) + b_c, \quad (4)$$

where w_c is the weight vector for class c . We remove negligible bias terms and only consider *Score* as the dot product between the feature vector and the weight vector. Furthermore, we can derive this from the geometric interpretation of vector inner products:

$$S_c = \|w_c\|_2 \cdot \|f(x)\|_2 \cdot \cos(\theta(f(x), w_c)). \quad (5)$$

The magnitude difference in Euclidean norm among each w_c is typically minimal, since a larger w_c indicates a prediction bias towards category c . Assuming all weight vectors w have uniform Euclidean norm, then x can be classified into the class that has the most similar weight vector to $f(x)$.

$$\begin{aligned} y(x) &= \underset{k}{\operatorname{argmax}}(\operatorname{sim}(f(x), w_c)), \\ \text{s.t. } \forall c &\in \{1, \dots, C\}, \end{aligned} \quad (6)$$

where $y(x)$ denotes the label of x , and $\operatorname{sim}()$ is a similarity measure function. From this perspective, w_c can be seen as the prototype for class c , which we refer to as the pseudo-prototype of c .

2) *Pseudo-Prototype-Based Feature Regularization*: While image augmentation readily employs semantic-preserving transformations (e.g., rotation, cropping) through direct pixel manipulation, feature-space augmentation requires fundamentally different approaches. Unlike image operations, semantic-consistent feature transformations must operate within the learned embedding space. Crucially, self-learning's pre-training on extensive labeled data yields a stable feature manifold, making it particularly amenable to feature-level augmentation.

In the learned feature space, class prototypes naturally serve as stable anchors for feature augmentation, where each prototype represents the centroid of its corresponding category's feature distribution. These prototypes provide the geometrically and statistically optimal reference points for feature manipulation due to their position at the density peaks of class-conditional distributions. The proposed approach formulates high-dimensional feature transformations as prototype-guided operations, where a boundary-aware loss function explicitly constrains the transformed features to remain within their original class decision regions. This mechanism of prototype-centric transformation and invariance enforcement ensures

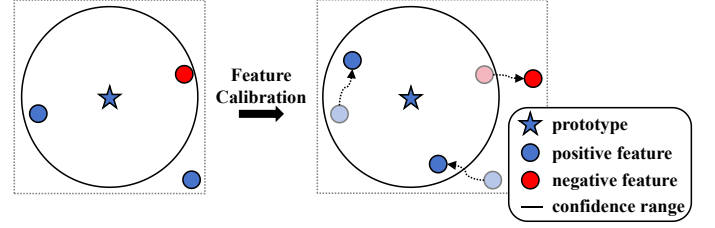


Fig. 4. Illustration of the feature calibration, where the model corrects "unfamiliar" features near the decision boundary in a pseudo-prototype based manner.

that augmented features both preserve semantic meaning and maintain discriminative power against other classes.

To achieve this, we employ KL divergence to constrain the consistency of the probability distributions before and after feature enhancement. The reason for choosing KL divergence lies in its ability to quantify the difference between two probability distributions, effectively ensuring that the enhanced features do not deviate from their original semantic meaning. By minimizing KL divergence, we can maintain the integrity of feature representation while enhancing the model's robustness and generalization capabilities. We define this loss function as:

$$\mathcal{L}_{\text{reg}} = \frac{1}{N} \sum_{j=1}^N \sum_{i=1}^C P_{\text{aug}}^j(i) \log \frac{P_{\text{aug}}^j(i)}{P^j(i)}, \quad (7)$$

where N represents the number of samples in the batch, $P^{(j)}(i)$ denotes the original probability of the i th class for the j th sample, and $P_{\text{aug}}^{(j)}(i)$ denotes the augmented probability of the i th class for the j th sample.

D. Pseudo-Prototype-Guided Feature Calibration

In self-training scenarios, since most unlabeled samples are previously unseen by the model, their feature patterns and distributions remain unfamiliar to the model, potentially leading to misclassification. To address this, we employ the pseudo-prototypes learned from labeled data as anchor points in the feature space. These prototypes enable feature calibration for unseen samples, effectively transforming their feature patterns into more model-recognizable representations through prototype-guided adaptation, as shown in Fig.4. This calibration process prior to pseudo-label assignment significantly mitigates erroneous label propagation while enhancing the model's decision-making accuracy.

In this section, we will detail how image features interact with pseudo-prototypes to enhance feature representations, as shown in Fig.5. Let the linear classifier be denoted as $\text{Clf}(\cdot)$, and the pseudo-prototype as $P \in R^{C \times d}$.

Apply a nonlinear transformation before obtaining the attention between the feature and the pseudo-prototype using matrix multiplication:

$$A = \operatorname{softmax}\left(\frac{Q(f(x)) \cdot K(P)}{\sqrt{d}}\right), \quad (8)$$

where Q and K represent the linear layers that transform features and pseudo-prototypes into query and key, respectively.

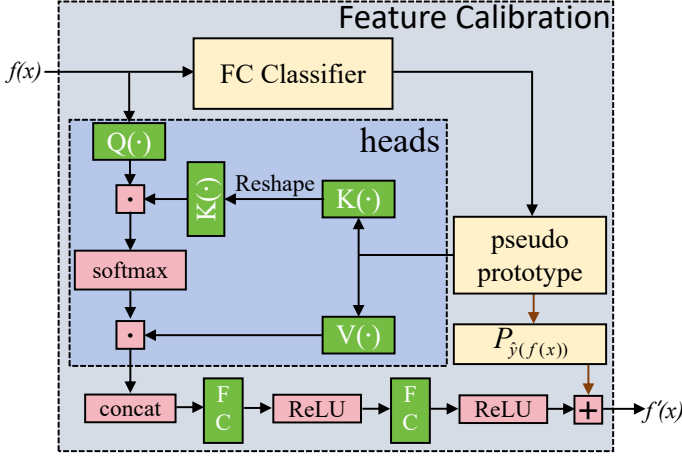


Fig. 5. The detailed process of the feature calibration module. The pseudo-prototype is derived from the weight matrix of the FC classifier, and the pseudo-prototype is used as K and V to perform interactive attention calculations on the input feature $f(x)$.

The attention is used to compute the weighted sum of features with the pseudo-prototypes by:

$$S = A \cdot V(P), \quad (9)$$

where V represent the linear layer that transforms pseudo-prototypes into values. In practice, we use multiple attention heads here, and we use the concat operation to merge the output of multiple heads.

Then, it is followed by two linear layers and an activation layer:

$$f_a = f_2(\text{ReLU}(f_1(\text{concat}(S_1, \dots, S_n)))), \quad (10)$$

here, f_1 and f_2 represent the two linear layers, and ReLU represents the ReLU activation function.

Finally, we apply a residual connection with its corresponding pseudo-prototype and conclude with the ReLU activation function:

$$f'(x) = \text{ReLU}(f_a + P_{\hat{y}(f(x))}). \quad (11)$$

It is worth noting that $\hat{y}(f(x))$ represents the predicted class of $f(x)$ by the model, i.e., the model's predicted label for the unenhanced features. At this point, we have obtained the enhanced features resulting from the interaction between $f(x)$ and the pseudo-prototype.

In the process of self-learning, we employ a class-balanced cross-entropy loss as the classification loss function and utilize pseudo-labels to guide the model's self-learning. Additionally, we incorporate a KL divergence loss to constrain the difference between the probability distributions of the original and augmented feature outputs. The entire loss we use in the self-learning process is as follows:

$$L_{self} = L_{CB-CE}(o) + L_{CB-CE}(o') + L_{reg}(o, o'), \quad (12)$$

where o represents the output probability of the original features after passing through the classifier $\text{Clf}(\cdot)$, while o'

represents the output probability of the augmented features after passing through $\text{Clf}(\cdot)$.

The loss function used in the main training process is generally similar to that in the self-learning process, with the primary difference being the use of the classification loss function. The optimization objective in the main training process is defined as follows:

$$L_{normal} = L_{CB}(o) + L_{CB}(o') + L_{reg}(o, o'). \quad (13)$$

In the main training process, we also leverage the output probability of the augmented features. This training allows the model to utilize a large amount of labeled data to facilitate the update and convergence of the feature calibration module.

IV. EXPERIMENTS

A. Dataset and setting

Dataset. We select CIFAR-100, tiny-ImageNet-200 datasets for image classification analysis. The CIFAR-100 dataset consists of 32×32 pixel color images across 100 classes, divided into 50,000 training and 10,000 test images. The tiny-ImageNet, a larger dataset, is designed to classify images into 200 categories and includes 100K training images along with 10K test images. To augment our data, we employ horizontal flipping and random cropping methods, resulting in output images of 224×224 pixels.

For the CIFAR-100 dataset, we combine the original 50,000 training images and 10,000 test images, and randomly shuffle them for further splitting. Specifically, a 70%:15%:15% split corresponds to 42,000 training images, 9,000 validation images, and 9,000 test images. On the other hand, a 60%:20%:20% split results in 36,000 training images, 12,000 validation images, and 12,000 test images. For the tiny-ImageNet dataset, we randomly shuffle all 110K images (combining the original 100K training images and 10K test images) and perform the split as follows: an 80%:10%:10% split corresponds to 87,843 training images, 10,983 validation images, and 11,174 test images.

Setting. We load pre-trained weights from ImageNet for all models, set the initial learning rate to $5e-3$, and use the SGD optimizer with a weight decay of $1e-4$. Additionally, we apply the OneCycleLR scheduler to dynamically adjust both the learning rate and momentum during training. The first 30% of the training phase is designated for learning rate warm-up, while the remaining 70% gradually reduces the learning rate using a cosine annealing strategy. We also enable momentum cycling, where the momentum increases from 0.85 to 0.95 and then gradually decreases.

B. Effectiveness of Mix-Supervised Learning

We conduct a series of comparative experiments to validate the effectiveness of mix-supervised learning in model training, comparing it with the traditional training approach. The models tested include four mainstream architectures: Convolutional Neural Networks (CNN), Transformer, MLP, and Graph Neural Networks (GNN), as well as the latest model, transNext. We set the batch size to 32 for all models.

TABLE I

The data in the table presents a comparison between the normal model training approach and our Mix-Supervised Learning approach on the CIFAR-100 dataset. All models are loaded with pre-trained weights. ♠CNN, ♣TRANSFORMER, ♦MLP, ★GNN.

train:val:test	70%:15%:15%				60%:20%:20%			
method	baseline		MSL		baseline		MSL	
Top-1 Accuracy	val_acc	test_acc	val_acc	test_acc	val_acc	test_acc	val_acc	test_acc
♠ResNet34 [9]	83.27±0.39	83.25±0.15	83.62±0.29	83.67±0.08	82.38±0.07	81.98±0.12	82.94±0.13	82.45±0.08
♠ResNet50 [9]	83.95±0.13	83.75±0.13	84.46±0.14	84.62±0.24	82.69±0.15	82.25±0.05	83.24±0.04	82.84±0.14
♠ResNet152 [9]	86.74±0.05	86.76±0.04	86.5±0.05	87.02±0.33	86.47±0.06	85.87±0.21	86.42±0.13	86.1±0.19
♠ResNeXt50 [40]	83.85±0.12	83.93±0.26	84.73±0.19	84.79±0.36	82.81±0.05	82.39±0.13	83.33±0.08	83.41±0.24
♠ResNeSt50 [44]	83.97±0.29	84.34±0.07	85.13±0.24	84.97±0.19	83.25±0.21	83.41±0.07	83.94±0.09	83.89±0.13
♠EfficientNet-b3 [30]	87.41±0.07	87.39±0.17	87.97±0.10	87.99±0.08	86.83±0.29	86.53±0.10	87.27±0.15	87.11±0.06
♠DenseNet121 [10]	83.73±0.04	83.85±0.19	83.88±0.13	83.91±0.17	83.15±0.11	82.73±0.04	83.11±0.05	83.18±0.04
♣PVT-s [37]	86.13±0.28	86.33±0.35	85.99±0.03	85.94±0.14	84.29±0.17	84.62±0.18	84.63±0.12	84.98±0.18
♣CVT-13 [39]	86.64±0.21	86.31±0.06	86.85±0.08	86.5±0.24	86.26±0.26	86.05±0.17	86.21±0.13	86.03±0.26
♣Swin-tiny [17]	86.93±0.14	86.82±0.09	87.88±0.03	87.46±0.19	85.43±0.22	85.53±0.20	86.53±0.18	86.79±0.19
♦Poolformer-s24 [43]	85.64±0.07	85.09±0.01	85.62±0.09	85.49±0.15	85.06±0.16	84.58±0.07	85.01±0.19	85.22±0.18
★Pvig-tiny [8]	84.11±0.02	83.78±0.12	84.77±0.14	84.2±0.13	81.82±0.11	81.49±0.03	83.16±0.05	82.77±0.07
♠transNext [27]	89.84±0.07	89.12±0.21	89.77±0.05	89.55±0.16	88.51±0.09	88.36±0.08	88.87±0.09	88.69±0.15

TABLE II

The data in the table presents a comparison between the normal model training approach and our Mix-Supervised Learning approach on the tiny-ImageNet dataset. All models are loaded with pre-trained weights. ♠CNN, ♣TRANSFORMER, ♦MLP, ★GNN.

train:val:test	80%:10%:10%				60%:20%:20%			
method	baseline		MSL		baseline		MSL	
Top-1 Accuracy	val_acc	test_acc	val_acc	test_acc	val_acc	test_acc	val_acc	test_acc
♠ResNet34 [9]	73.24±0.12	72.65±0.36	74.22±0.41	73.41±0.23	71.95±0.08	71.52±0.09	72.16±0.07	71.53±0.06
♠ResNet50 [9]	74.93±0.07	74.41±0.13	75.56±0.14	75.14±0.26	73.95±0.08	73.68±0.15	74.55±0.14	74.27±0.04
♠ResNet152 [9]	82.81±0.10	82.65±0.18	82.85±0.09	83.03±0.29	82.84±0.26	82.59±0.09	82.97±0.11	82.69±0.04
♠ResNeXt50 [40]	74.44±0.21	74.67±0.34	74.67±0.34	75.16±0.32	74.11±0.18	73.29±0.07	74.42±0.08	73.53±0.04
♠ResNeSt50 [44]	76.16±0.14	75.93±0.23	76.96±0.16	76.31±0.15	75.42±0.11	75.01±0.03	75.43±0.05	75.12±0.05
♠EfficientNet-b3 [30]	82.1±0.12	81.67±0.23	82.36±0.17	82.22±0.21	81.54±0.11	80.82±0.07	81.78±0.01	81.48±0.13
♠DenseNet121 [10]	74.29±0.23	73.76±0.22	74.97±0.24	74.19±0.22	73.76±0.09	73.09±0.07	73.5±0.05	73.16±0.06
♣PVT-s [37]	77.76±0.05	77.04±0.20	78.5±0.07	77.96±0.21	77.36±0.09	76.06±0.31	76.3±0.13	75.87±0.18
♣CVT-13 [39]	84.21±0.1	84.09±0.27	84.02±0.11	83.86±0.15	84.17±0.11	84.08±0.12	83.86±0.05	83.44±0.06
♣Swin-tiny [17]	81.45±0.17	81.21±0.08	82.17±0.09	81.81±0.15	80.54±0.22	80.21±0.04	81.23±0.13	81.02±0.06
♦Poolformer-s24 [43]	81.64±0.12	81.79±0.13	82.36±0.08	82.24±0.04	81.92±0.05	81.21±0.07	81.42±0.05	81.14±0.26
★Pvig-tiny [8]	75.26±0.13	75.01±0.13	76.03±0.67	75.61±0.88	74.93±0.16	74.32±0.17	75.46±0.19	74.53±0.18
♠transNext [27]	85.43±0.21	84.76±0.31	85.74±0.15	84.91±0.19	84.83±0.16	84.58±0.11	85.09±0.09	84.74±0.02

In addition to evaluating model performance on the test set, we also assess performance on the validation set to observe whether the model learn correct knowledge through mix-supervised learning. Furthermore, we explore the model's performance on datasets with different split ratios. It is worth noting that we focus more on the model's performance on the test set. Only the highest test accuracy is highlighted in bold, as test performance more accurately reflects the model's practical effectiveness.

As shown in Table I and Table II, the mix-supervised learning approach generally achieves better test performance in most cases, indicating that mix-supervised learning can enhance the model's classification accuracy on unseen data. In addition, we also observe that under the distribution with a lower proportion of training data (i.e., 60%:20%:20%), the model's performance is generally worse compared to configurations with a higher proportion of training data. This aligns with our expectations, as the model has less train-

ing data and thus learns less knowledge. Moreover, it is noteworthy that models trained with mix-supervised learning exhibit marginally superior performance on the validation set compared to the baseline, though the improvement is not statistically significant. This observation suggests that mix-supervised learning allows the model to extract a limited yet meaningful amount of knowledge from the validation set while crucially avoiding overfitting. Consequently, the fundamental role of the validation set— unbiased evaluation— remains intact.

C. Ablation Study

1) *Mix-Supervised Learning*: We perform ablation experiments to analyze the effects of *Pseudo-prototype guided Feature Calibration*, *Class Balanced Cross Entropy Loss*, and *Model Rewind*, which are used in the mix-supervised learning process, to investigate their individual impacts on the model's classification accuracy. The experiment is conducted under

the 80%:10%:10% split ratio of the tiny-ImageNet dataset, using the ResNet50 model. As shown in Table III, *Calibration*

TABLE III
ABLATION STUDY IN MIX-SUPERVISED LEARNING.

Calibration	✓			✓		✓	✓
CB-CE Loss		✓		✓	✓		✓
Rewind			✓		✓	✓	✓
val_acc	75.36	75.21	75.18	75.5	75.28	75.54	75.65
test_acc	75.07	74.85	74.5	75.21	74.88	75.04	75.25

has the most significant impact on improving the model’s accuracy, followed by *CB-CE Loss*, while *Rewind* has the least effect. When all three components are applied together, they yield the greatest improvement in model accuracy, followed by the combination of *Calibration* and *CB-CE Loss*. Firstly, *Calibration* enhances the expressive power of features through interaction with pseudo-prototypes, which explains its relatively large contribution. Secondly, *CB-CE Loss* addresses the issue of class imbalance in pseudo-label assignments, helping to mitigate class bias in model predictions, thereby improving classification accuracy. Lastly, the primary purpose of *Rewind* is to guide the model toward the correct convergence direction, so its direct impact on improving classification accuracy is not significant.

2) *Main Training Process*: We also analyze the impact of *CE Loss*, *Pseudo-prototype guided Feature Calibration*, and *Pseudo-prototype-based Feature Regularization* on the model’s classification accuracy during the main training process. The experiments are conducted under the 80%:10%:10% split ratio of the tiny-ImageNet dataset, using the ResNet50 model.

TABLE IV
ABLATION STUDY IN MAIN TRAINING PROCESS.

CE Loss	✓		✓		✓	✓
Calibration			✓			✓
Regularization				✓		✓
val_acc	75.09±0.08	75.23±0.15	75.41±0.03	75.71±0.12		
test_acc	74.84±0.28	74.97±0.22	75.26±0.25	75.38±0.11		

As shown in Table IV, the ablation study results demonstrate the incremental improvements achieved by progressively adding components to the baseline model using only cross-entropy (CE) loss, which yields validation and test accuracies of 75.09% and 74.84% respectively with relatively small variances (± 0.08 and ± 0.28), indicating stable model performance. Incorporating calibration brings modest but consistent gains of 0.14% and 0.13% in validation and test accuracies (75.23% and 74.97%), while adding regularization alone shows more substantial improvements of 0.32% and 0.42% over the baseline (75.41% and 75.26%), suggesting that regularization has a stronger individual impact on model performance than calibration. The full model combining all components achieves the best performance with 75.71% validation accuracy and 75.38% test accuracy, representing significant gains of 0.62% and 0.54% respectively over the baseline, while maintaining low variance (± 0.12 and ± 0.11), which indicates that the synergistic combination of CE loss, calibration, and regularization

leads to both higher accuracy and more robust performance compared to using any component in isolation.

D. Analysis of the hyperparameters

1) *Feature Regularization*: The hyperparameter lambda is used to control the influence of the feature regularization loss. To analyze the impact of feature regularization loss on model performance during training, we test six different values of lambda using the ResNet50 model on the tiny-ImageNet dataset, with two different data splits: 6:2:2 and 8:1:1.

As shown in TABLE V, firstly, we find that the model accuracy under the 8:1:1 split is generally higher than that under the 6:2:2 split, as a higher proportion of the dataset is allocated for training, leading to improved model performance. Secondly, we observe that the model performs the worst when lambda is set to 3. As lambda decreases from 3 to 0.1, the model’s performance gradually improves, suggesting that feature regularization loss plays a more auxiliary role during training. Mild feature regularization can enhance model performance, while overly high regularization negatively impacts it. Finally, when lambda is set to 0, the model’s classification accuracy is slightly lower, indicating that feature regularization has a meaningful impact on improving model classification performance.

TABLE V
ANALYSIS OF THE HYPERPARAMETER LAMBDA ON THE TINY-IMAGENET DATASET UNDER TWO DIFFERENT DATA SPLIT RATIOS.

lambda\ratio	6:2:2		8:1:1	
	val_acc	test_acc	val_acc	test_acc
3	71.1±0.09	71.03±0.2	72.77±0.1	72.03±0.17
1	73.48±0.08	73±0.02	74.55±0.09	74.42±0.11
0.5	74.13±0.06	73.56±0.05	75.57±0.11	74.6±0.07
0.3	74.35±0.08	73.85±0.09	75.41±0.1	75.02±0.17
0.1	74.54±0.10	74.13±0.21	76.03±0.28	75.22±0.22
0	74.24±0.05	73.68±0.16	74.91±0.04	74.43±0.15

2) *Model Rewind*: Model rewind aims to guide the model in the correct direction for gradient updates. In this section, we explore the impact of different threshold values for triggering the model’s rewind and the frequency of triggers on model performance, as shown in TABLE VI. Before the model performs self-learning, we record an accuracy metric A_{before} on the validation set. After the model performs self-learning, we evaluate the model again on the validation set to obtain a new accuracy metric A_{after} . If the difference between $A_{\text{after}} - A_{\text{before}}$ is smaller than a preset threshold, we consider that the update direction of the self-learning is incorrect and trigger a rewind. We select five different threshold values and train the ResNet50 model on the CIFAR-100 dataset with a 6:2:2 split ratio. The recorded results are given as the average and standard deviation of three repeated experiments.

First, it is logical that as the threshold decreases, the frequency of rewind triggers increases. Notably, when the threshold is set to 1, the number of rewind triggers significantly rises. Second, decreasing the threshold consistently improves validation and test accuracy, with peak performance occurring at a threshold of 1 - coinciding with maximum rewind triggers.

TABLE VI
ANALYSIS OF THE TRIGGER COUNT AND ACCURACY PERFORMANCE
CAUSED BY DIFFERENT TRIGGER THRESHOLDS OF REWIND IN 3
REPEATED EXPERIMENTS.

rewind threshold	val_acc	test_acc	rewind counts
5	84.52±0.11	83.91±0.24	1
4	84.36±0.20	83.76±0.17	2
3	84.31±0.16	84.14±0.02	4
2	84.32±0.11	84.10±0.33	5
1	84.47±0.20	84.22±0.04	13

This strongly validates our rewind mechanism’s effectiveness in correcting erroneous update directions during self-learning.

3) *Degree of Confidence*: In our self-learning process, the allocation of pseudo-labels is entirely based on the model’s classification confidence. In this section, we analyze the number and accuracy of the pseudo-labels assigned at different confidence levels, as shown in TABLE VII. We conduct experiments using the ResNet50 model on the CIFAR-100 dataset with a 6:2:2 split ratio, training for a total of 30 epochs, and recording results at the 10th and 20th epochs.

TABLE VII
ANALYSIS OF THE NUMBER OF PSEUDO-LABELS ASSIGNED AND
RELIABILITY UNDER DIFFERENT CONFIDENCE LEVELS IN
SELF-LEARNING.

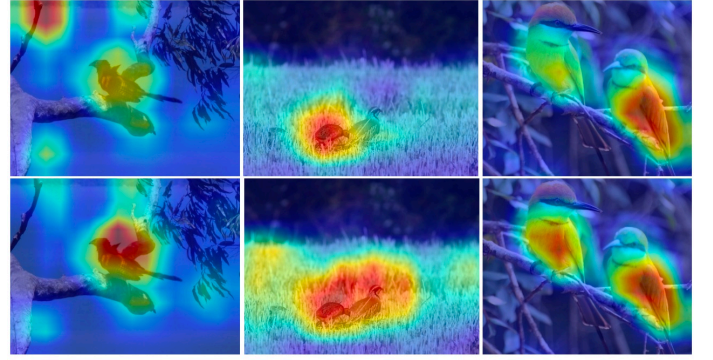
confidence epoch	0.90		0.95		0.99	
	10	20	10	20	10	20
pseudo counts	3679	4877	2755	3912	1671	2838
correct counts	3470	4621	2688	3784	1656	2799
correctness	0.943	0.948	0.976	0.967	0.991	0.986

First, as expected, with an increase in confidence, the number of pseudo-labels assigned gradually decreases, while the accuracy of the assigned pseudo-labels increases correspondingly. Second, we observe that regardless of the confidence level, the number of pseudo-labels assigned at the 20th epoch was higher than at the 10th epoch. This is because, as training progresses and the model gradually converges, its classification ability improves, resulting in more confident predictions. Last, we were somewhat surprised to find that the accuracy of the assigned pseudo-labels does not significantly improve as the model converges. This suggests that while the model’s classification ability increases over time, it also attempts to assign more pseudo-labels, which prevents a noticeable improvement in accuracy.

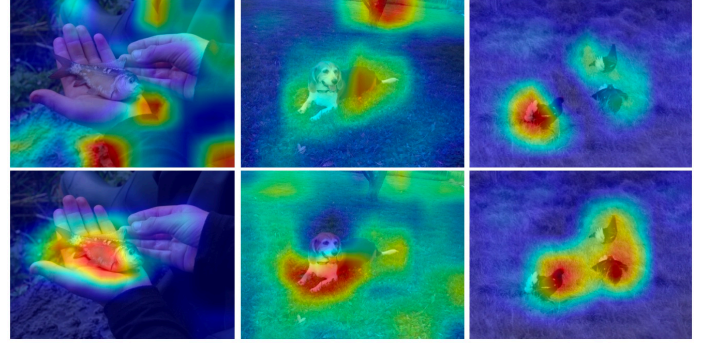
E. Visualization

In this section, we utilize explainable visualization results to demonstrate the enhancement of model generalization performance through mix-supervised learning. We employ the Grad-CAM technique [25] with a ResNet50 model. The experiment involves training the model on the CUB-200 dataset and performing visualizations on the ImageNet-1K dataset. This approach is designed to evaluate the model’s performance on “unknown data” beyond the “known data.”

From the In-Domain Cross-Dataset section of Fig. 6, it can be observed that the model trained with mix-supervised



In-Domain Cross-Dataset



Out-of-Domain Cross-Dataset

Fig. 6. Class Activation Map comparisons between baseline and mix-supervised learning approaches. The visualization results of 3 randomly selected images from both in-domain (bird category) and out-of-domain (non-bird category) subsets of ImageNet-1K, after training on the CUB-200 dataset. Top: baseline results; Bottom: mix-supervised learning results.

learning more accurately localizes the positions of bird objects and performs better in scenarios with multiple birds. Notably, in the first image, the baseline model’s attention is more focused on the tree branch in the upper left corner. This may be because the model has not encountered this type of bird before but has frequently seen tree branches, causing its attention to be drawn to the background. Furthermore, from the Out-of-Domain Cross-Dataset section of Fig. 6, it is evident that for object types never seen before, the baseline model almost fails to notice the objects, whereas the model using mix-supervised learning better focuses on the objects. This experiment demonstrates the positive effect of the mix-supervised learning method in improving model generalization. This improvement stems from the mix-supervised learning approach, which more fully utilizes the available data, and the feature calibration method also corrects the model’s unseen feature patterns.

F. Class Imbalance in Pseudo-Labeling

In this section, we explore the issue of class imbalance in the assignment of pseudo-labels during the self-learning process. We also justify the design of the CB-CE Loss function and demonstrate its effectiveness. We conduct experiments on the CIFAR-100 dataset, but for ease of observation, we group the classes into 20 superclasses. The ResNet50 model is used for the experiments, which are run for a total of 30 epochs. In

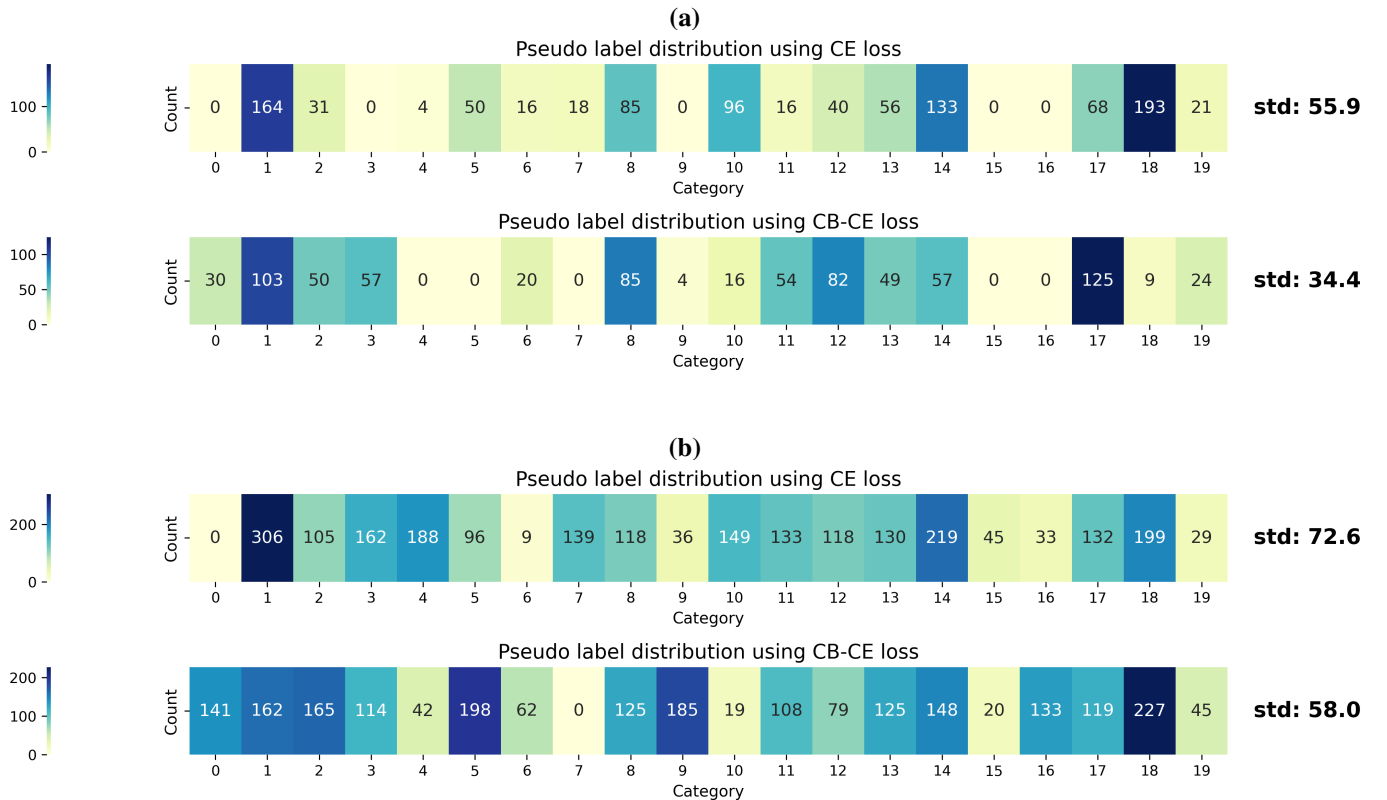


Fig. 7. The heatmaps of the pseudo-label distribution across categories in the training. We compare the pseudo-label distribution results using traditional CE loss and CB-CE loss, conducted on the 20 superclasses of the CIFAR-100 dataset, and show the standard deviation. (a) illustrates the pseudo-label distributions in the early training phase, while (b) demonstrates the mid-training phase distributions.

Fig.7(a) and Fig.7(b), we show the distribution of pseudo-labels across different classes at the 5th and 10th epochs, respectively, along with the standard deviation of pseudo-label distribution on the right side.

As shown in Fig.7(a), when using the traditional cross-entropy loss function for training, at the 5th epoch, the model assigns 164, 133, and 193 pseudo-labels to classes 1, 14, and 18, respectively, while no pseudo-labels were assigned to classes 0, 3, 4, 9, 15, and 16. The standard deviation of the pseudo-label distribution across all classes is 55.9. In contrast, when using CB-CE Loss, only classes 1 and 17 have pseudo-label assignments greater than 100, and the standard deviation of pseudo-label distribution across all classes is 34.4. Fig.7(b) shows the distribution of pseudo-labels at the 10th epoch. When using the traditional cross-entropy loss, the model assigns 306 pseudo-labels to class 1, with a standard deviation of 72.6. However, with CB-CE Loss, the highest number of pseudo-labels assigned is 227, with a standard deviation of 58.0.

First, a simple comparison of the standard deviations clearly shows that CB-CE Loss significantly alleviates the class imbalance issue in pseudo-label distribution. Second, upon inspecting specific data points, we can observe that the model guided by CB-CE Loss does not assign an excessive number of pseudo-labels to individual classes. Most critically, when using traditional cross-entropy loss, the top three classes by pseudo-label count at epoch 5 (classes 1, 14, and 18) remain

dominant at epoch 10, indicating persistent class imbalance.

These results demonstrate that models trained with cross-entropy loss exhibit persistent class preference in pseudo-label assignment, maintaining initial bias throughout training. This fundamentally reflects cross-entropy’s inability to counteract class imbalance. In contrast, CB-CE Loss effectively addresses both the imbalance in pseudo-label distribution.

V. CONCLUSIONS

This paper introduces the mix-supervised learning, a novel training paradigm that synergizes supervised and unsupervised learning principles inspired by human cognitive processes. The paradigm enables models to extract knowledge from distributionally-shifted “unseen” samples through several key innovations: Class-Balanced Cross-Entropy (CB-CE) Loss to mitigate pseudo-labeling bias; Model Rewind for erroneous update correction; Pseudo-Prototype Feature Regularization ensuring semantic-consistent feature transformations, coupled with Pseudo-Prototype Guided Calibration to enhance pseudo-label reliability. Finally, experimental results on the CIFAR-100 and tiny-ImageNet datasets demonstrate that the model learns more knowledge and improves its generalization ability after undergoing mix-supervised learning. This work opens new avenues for developing more human-like learning systems that can continuously adapt to evolving data distributions.

REFERENCES

- [1] Relja Arandjelovic, Petr Gronat, Akihiko Torii, Tomas Pajdla, and Josef Sivic. Netvlad: Cnn architecture for weakly supervised place recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5297–5307, 2016.
- [2] Haiyan Chen, Yizhen Jia, Jiaming Ge, and Bin Gu. Incremental learning algorithm for large-scale semi-supervised ordinal regression. *Neural Networks*, 149:124–136, 2022.
- [3] Chun-Te Chu and Jenq-Neng Hwang. Fully unsupervised learning of camera link models for tracking humans across nonoverlapping cameras. *IEEE transactions on circuits and systems for video technology*, 24(6):979–994, 2014.
- [4] Lieven De Lathauwer, Bart De Moor, and Joos Vandewalle. A multi-linear singular value decomposition. *SIAM journal on Matrix Analysis and Applications*, 21(4):1253–1278, 2000.
- [5] Fadi Dornaika, Jingjun Bi, and Chongsheng Zhang. A unified deep semi-supervised graph learning scheme based on nodes re-weighting and manifold regularization. *Neural Networks*, 158:188–196, 2023.
- [6] Fernando P Dos Santos, Cemre Zor, Josef Kittler, and Moacir A Ponti. Learning image features with fewer labels using a semi-supervised deep convolutional network. *Neural Networks*, 132:131–143, 2020.
- [7] Rui Guo, Jie Sun, Chencheng Zhang, and Xiaohua Qian. A self-supervised metric learning framework for the arising-from-chair assessment of parkinsonians with graph convolutional networks. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(9):6461–6471, 2022.
- [8] Kai Han, Yunhe Wang, Jianyuan Guo, Yehui Tang, and Enhua Wu. Vision gnn: An image is worth graph of nodes. *Advances in neural information processing systems*, 35:8291–8303, 2022.
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [10] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.
- [11] Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. arxiv 2018. *arXiv preprint arXiv:1812.04948*, 1812.
- [12] Durk P Kingma, Shakir Mohamed, Danilo Jimenez Rezende, and Max Welling. Semi-supervised learning with deep generative models. *Advances in neural information processing systems*, 27, 2014.
- [13] Teuvo Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990.
- [14] Gabriele Lagani, Fabrizio Falchi, Claudio Gennaro, and Giuseppe Amato. Hebbian semi-supervised learning in a sample efficiency setting. *Neural Networks*, 143:719–731, 2021.
- [15] Bo Li, Zhipeng Yang, Zhuoran Jia, and Hao Ma. An unsupervised learning neural network for planning uav full-area reconnaissance path. *Xibei Gongye Daxue Xuebao/Journal of Northwestern Polytechnical University*, 39(1):77–84, 2021.
- [16] Lin Li, Hongchun Qu, Zhaoni Li, Jian Zheng, and Fei Guo. Discriminative projection learning with adaptive reversed graph embedding for supervised and semi-supervised dimensionality reduction. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(12):8688–8702, 2022.
- [17] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021.
- [18] Ying Lv, Zhi Liu, and Gongyang Li. Context-aware interaction network for rgb-t semantic segmentation. *IEEE Transactions on Multimedia*, 26:6348–6360, 2024.
- [19] Andrew McCallum, Kamal Nigam, et al. A comparison of event models for naive bayes text classification. In *AAAI-98 workshop on learning for text categorization*, volume 752, pages 41–48. Madison, WI, 1998.
- [20] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fiedel, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
- [21] Curtis Northcutt, Lu Jiang, and Isaac Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021.
- [22] Fang Peng, Xiaoshan Yang, Linhui Xiao, Yaowei Wang, and Changsheng Xu. Sgva-clip: Semantic-guided visual adapting of vision-language models for few-shot image classification. *IEEE Transactions on Multimedia*, 26:3469–3480, 2023.
- [23] Kristopher J Preacher, Patrick J Curran, and Daniel J Bauer. Computational tools for probing interactions in multiple linear regression, multilevel modeling, and latent curve analysis. *Journal of educational and behavioral statistics*, 31(4):437–448, 2006.
- [24] Alec Radford. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015.
- [25] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: visual explanations from deep networks via gradient-based localization. *International journal of computer vision*, 128:336–359, 2020.
- [26] Heng Tao Shen. Principal component analysis. *Encyclopedia of database systems*, 2136, 2009.
- [27] Dai Shi. Transnext: Robust foveal visual perception for vision transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17773–17783, 2024.
- [28] Hoo-Chang Shin, Holger R Roth, Mingchen Gao, Le Lu, Ziyue Xu, Isabella Nogueira, Jianhua Yao, Daniel Mollura, and Ronald M Summers. Deep convolutional neural networks for computer-aided detection: Cnn architectures, dataset characteristics and transfer learning. *IEEE transactions on medical imaging*, 35(5):1285–1298, 2016.
- [29] Kihyuk Sohn, David Berthelot, Nicholas Carlini, Zizhao Zhang, Han Zhang, Colin A Raffel, Ekin Dogus Cubuk, Alexey Kurakin, and Chun-Liang Li. Fixmatch: Simplifying semi-supervised learning with consistency and confidence. *Advances in neural information processing systems*, 33:596–608, 2020.
- [30] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.
- [31] Xiangyuan Tan, Xiaoguang Gao, Zidong Wang, Hao Han, Xiaohan Liu, and Daqing Chen. Learning the structure of bayesian networks with ancestral and/or heuristic partition. *Information Sciences*, 584:719–751, 2022.
- [32] Enmei Tu, Zihao Wang, Jie Yang, and Nikola Kasabov. Deep semi-supervised learning via dynamic anchor graph embedding in latent space. *Neural Networks*, 146:350–360, 2022.
- [33] Arash Vahdat and Jan Kautz. Nvae: A deep hierarchical variational autoencoder. *Advances in neural information processing systems*, 33:19667–19679, 2020.
- [34] Jesper E Van Engelen and Holger H Hoos. A survey on semi-supervised learning. *Machine learning*, 109(2):373–440, 2020.
- [35] Kaifang Wan, Dingwei Wu, Yiwei Zhai, Bo Li, Xiaoguang Gao, and Zijian Hu. An improved approach towards multi-agent pursuit–evasion game decision-making using deep reinforcement learning. *Entropy*, 23(11):1433, 2021.
- [36] Li Wang, Xinyao Yang, Haoyue Tan, Xueru Bai, and Feng Zhou. Few-shot class-incremental sar target recognition based on hierarchical embedding and incremental evolutionary network. *IEEE Transactions on Geoscience and Remote Sensing*, 61:1–11, 2023.
- [37] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 568–578, 2021.
- [38] Xiangwen Wang, Xianghong Lin, and Xiaochao Dang. Supervised learning in spiking neural networks: A review of algorithms and evaluations. *Neural Networks*, 125:258–280, 2020.
- [39] Haiping Wu, Bin Xiao, Noel Codella, Mengchen Liu, Xiyang Dai, Lu Yuan, and Lei Zhang. Cvt: Introducing convolutions to vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 22–31, 2021.
- [40] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1492–1500, 2017.
- [41] Qin Xu, Jiahui Wang, Bo Jiang, and Bin Luo. Fine-grained visual classification via internal ensemble learning transformer. *IEEE Transactions on Multimedia*, 25:9015–9028, 2023.
- [42] Rui Xu, Lei Xing, Shuai Shao, Lifu Zhao, Baodi Liu, Weifeng Liu, and Yicong Zhou. Gct: Graph co-training for semi-supervised few-shot learning. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(12):8674–8687, 2022.

- [43] Weihao Yu, Mi Luo, Pan Zhou, Chenyang Si, Yichen Zhou, Xinchao Wang, Jiashi Feng, and Shuicheng Yan. Metaformer is actually what you need for vision. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10819–10829, 2022.
- [44] Hang Zhang, Chongruo Wu, Zhongyue Zhang, Yi Zhu, Haibin Lin, Zhi Zhang, Yue Sun, Tong He, Jonas Mueller, R Manmatha, et al. Resnest: Split-attention networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2736–2746, 2022.
- [45] Hongwei Zhang, Guoqing Zhang, Yuhao Chen, and Yuhui Zheng. Global relation-aware contrast learning for unsupervised person re-identification. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(12):8599–8610, 2022.
- [46] Huang Zhang, Changshuo Wang, Long Yu, Shengwei Tian, Xin Ning, and Joel Rodrigues. Pointgt: A method for point-cloud classification and segmentation based on local geometric transformation. *IEEE Transactions on Multimedia*, 2024.
- [47] Kai Zhang, Wangmeng Zuo, Yunjin Chen, Deyu Meng, and Lei Zhang. Beyond a gaussian denoiser: Residual learning of deep cnn for image denoising. *IEEE transactions on image processing*, 26(7):3142–3155, 2017.
- [48] Shizhou Zhang, Dexuan Kong, Yinghui Xing, Yue Lu, Lingyan Ran, Guoqiang Liang, Hexu Wang, and Yanning Zhang. Frequency-guided spatial adaptation for camouflaged object detection. *IEEE Transactions on Multimedia*, 27:72–83, 2025.
- [49] Yongyue Zhang, Michael Brady, and Stephen Smith. Segmentation of brain mr images through a hidden markov random field model and the expectation-maximization algorithm. *IEEE transactions on medical imaging*, 20(1):45–57, 2001.
- [50] Lin Zhao, Hui Zhou, Xinge Zhu, Xiao Song, Hongsheng Li, and Wenbing Tao. Lif-seg: Lidar and camera image fusion for 3d lidar semantic segmentation. *IEEE Transactions on Multimedia*, 26:1158–1168, 2023.
- [51] Menghua Zheng, Keyan Zhi, Jiawen Zeng, Chunwei Tian, and Lei You. A hybrid cnn for image denoising. *Journal of Artificial Intelligence and Technology*, 2(3):93–99, 2022.
- [52] Tinghui Zhou, Matthew Brown, Noah Snavely, and David G Lowe. Un-supervised learning of depth and ego-motion from video. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1851–1858, 2017.
- [53] Xiaojin Zhu, Zoubin Ghahramani, and John D Lafferty. Semi-supervised learning using gaussian fields and harmonic functions. In *Proceedings of the 20th International conference on Machine learning (ICML-03)*, pages 912–919, 2003.
- [54] Zhaorui Zhu and Quanxue Gao. Semi-supervised clustering via cannot link relationship for multiview data. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(12):8744–8755, 2022.



Chenyang Wu received the bachelor's degree in engineering from Guizhou University, Guizhou, China. His research interests include model acceleration, computer vision, artificial intelligence, video processing and generation.



Jiebo Luo (Fellow, IEEE) received the B.S. and M.S. degrees in electrical engineering from the University of Science and Technology of China, Hefei, respectively, and the Ph.D. degree in electrical engineering from the University of Rochester, Rochester, NY. Currently, he is on the editorial boards of the *IEEE Transactions on Multimedia*, *Pattern Recognition*, and the *Journal of Electronic Imaging*, as well as a Co-Chair of the 2007 SPIE International Symposium on Visual Communication and Image Processing. His research interests include

image processing, pattern recognition, computer vision, medical imaging, and multimedia communication.



Qinglei Li is currently pursuing the master's degree with the College of Computer Science and Technology, Guizhou University, Guiyang, China. His research interests include artificial intelligence, computer vision, image classification, image processing, and feature representation.



Nanyi Wang is currently pursuing the master's degree with the College of Computer Science and Technology, Guizhou University, Guiyang, China. Her research interests include artificial intelligence, computer vision, image classification, image processing, and Semi-supervised algorithm learning.



Qi Wang (Member, IEEE) is currently a Special Term Professor in State Key Laboratory of Public Big Data, Guizhou University. He received Ph.D. degree in Computer Application Engineering at School of Computer Science and Technology, Guangdong University of Technology, Guangdong, China, in 2020. He also received a Ph.D. degree in Engineering Technology at Faculty of Engineering and Technology, Hasselt University, Hasselt, Belgium, in 2021. His current research interests include computer vision, intelligent agriculture.